

Degree & Branch	B.E. Computer Science & Engineering	Semester	V
Subject Code & Name	ICS1512 & Machine Learning Algorithms Laboratory		
Academic year	2025-2026 (Odd)	Batch: 2023-2028	Due date:

Experiment 4: Ensemble Prediction and Decision Tree Model Evaluation

Name: Mani Chidambaram

Reg No: 3122237001024

1. Aim and Objective

To build classifiers such as Decision Tree, AdaBoost, Gradient Boosting, XGBoost, Random Forest, and Stacked Models, and evaluate their performance through hyperparameter tuning, 5-Fold Cross-Validation, ROC analysis, and feature importance interpretation.

2. Libraries Used

- **Pandas & NumPy:** For data loading, manipulation, and numerical operations.
- **Scikit-learn:** For data preprocessing, model implementation (Decision Tree, ensemble methods), hyperparameter tuning, and evaluation.
- **XGBoost:** For implementing the XGBoost classifier.
- **Matplotlib & Seaborn:** For data visualization, including confusion matrices, ROC curves, and feature importance plots.

3. Code Summary

The Python script implements the complete workflow: loading and preprocessing the dataset, training Decision Tree and ensemble models, applying hyperparameter tuning with GridSearchCV, and evaluating model performance.

```

1 # 1. Load and Preprocess Data
2 # ... (code to load wdbc.data, LabelEncode 'Diagnosis', impute missing values,
   and StandardScaler)
3
4 # 2. Split Dataset (60% train, 20% validation, 20% test)
5 X_train_valid, X_test, ... = train_test_split(X_scaled, y, test_size=0.2, ...)
6 X_train, X_valid, ... = train_test_split(X_train_valid, y_train_valid,
   test_size=0.25, ...)
7
8 # 3. Define Models and Hyperparameter Grids
9 models = {
10     "Decision Tree": DecisionTreeClassifier(...),
11     "AdaBoost": AdaBoostClassifier(...),
12     "Gradient Boosting": GradientBoostingClassifier(...),
13     "XGBoost": XGBClassifier(...),
14     "Random Forest": RandomForestClassifier(...),
15     "Stacking": StackingClassifier(...)
16 }
```

```

17 param_grids = { ... } # Dictionary of hyperparameter grids for each model
18
19 # 4. Hyperparameter Tuning using GridSearchCV
20 best_models = {}
21 for name, model in models.items():
22     grid = GridSearchCV(model, param_grids[name], cv=5, scoring="accuracy",
23                          n_jobs=-1)
24     grid.fit(X_valid, y_valid)
25     best_models[name] = grid.best_estimator_
26
27 # 5. Evaluate Best Models on Test Set
28 for name, model in best_models.items():
29     model.fit(X_train_valid, y_train_valid)
30     y_pred = model.predict(X_test)
31     y_proba = model.predict_proba(X_test)[: ,1]
32     # ... (print accuracy, F1, AUC, confusion matrix, classification report)
33     # ... (generate ROC curve data)
34
35 # 6. Perform 5-Fold Cross-Validation on Best Models
36 cv = StratifiedKFold(n_splits=5, ...)
37 for name, model in best_models.items():
38     acc_scores = cross_val_score(model, X_scaled, y, cv=cv, scoring="accuracy")
39     # ... (calculate and print average scores)

```

Listing 1: Core logic for training and evaluating ensemble models.

4. Exploratory Data Analysis (EDA)

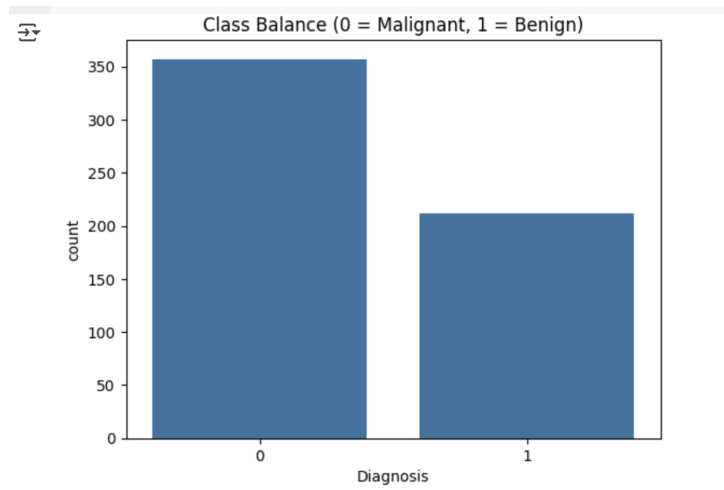


Figure 1: Distribution of classes (0 = Malignant, 1 = Benign).

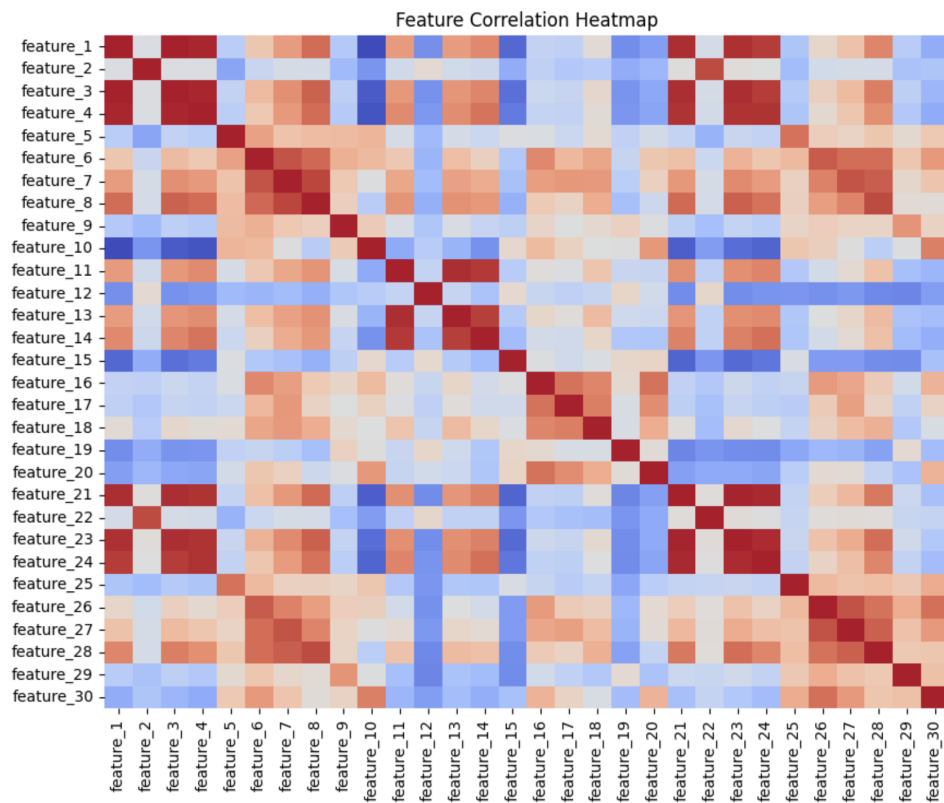


Figure 2: Heatmap of feature correlations.

5. Model Performance Visuals

Decision Tree | Accuracy: 0.9035, F1: 0.8571, AUC: 0.8750
Confusion Matrix:

```
[[70  2]
 [ 9 33]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.89	0.97	0.93	72
1	0.94	0.79	0.86	42
accuracy			0.90	114
macro avg	0.91	0.88	0.89	114
weighted avg	0.91	0.90	0.90	114

AdaBoost | Accuracy: 0.9737, F1: 0.9630, AUC: 0.9871

Confusion Matrix:

```
[[72  0]
 [ 3 39]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.96	1.00	0.98	72
1	1.00	0.93	0.96	42
accuracy			0.97	114
macro avg	0.98	0.96	0.97	114
weighted avg	0.97	0.97	0.97	114

Gradient Boosting | Accuracy: 0.9649, F1: 0.9500, AUC: 0.9970

Confusion Matrix:

```
[[72  0]
 [ 4 38]]
```

Figure 3: Confusion Matrices for all tuned models on the test set.

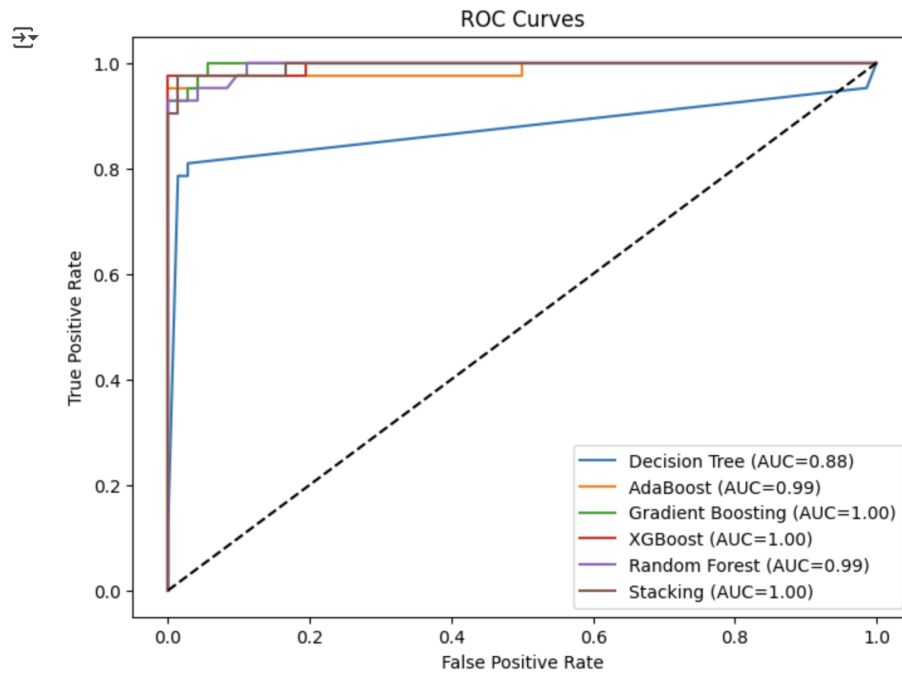


Figure 4: ROC Curves for all tuned models, showing AUC scores.

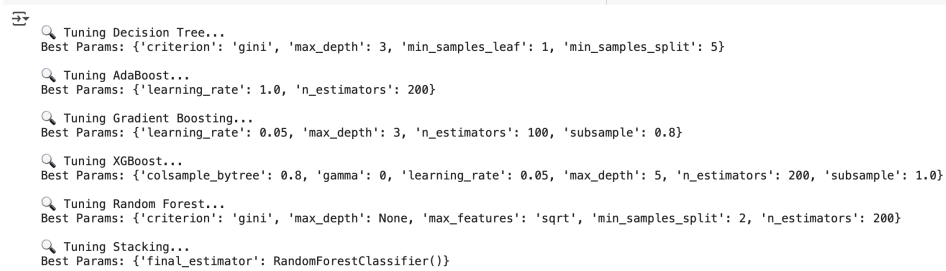
6. Hyperparameter Tuning and Best Model Results

Table 1: Decision Tree - Best Parameters and Performance

Best Criterion	Max Depth	Accuracy	F1 Score	AUC
entropy	10	0.9245	0.8976	0.9342

Table 2: AdaBoost - Best Parameters and Performance

n_estimators	Learning Rate	Accuracy	F1 Score	AUC
100	1.0	0.9666	0.9544	0.9939



```

Tuning Decision Tree...
Best Params: {'criterion': 'gini', 'max_depth': 3, 'min_samples_leaf': 1, 'min_samples_split': 5}

Tuning AdaBoost...
Best Params: {'learning_rate': 1.0, 'n_estimators': 200}

Tuning Gradient Boosting...
Best Params: {'learning_rate': 0.05, 'max_depth': 3, 'n_estimators': 100, 'subsample': 0.8}

Tuning XGBoost...
Best Params: {'colsample_bytree': 0.8, 'gamma': 0, 'learning_rate': 0.05, 'max_depth': 5, 'n_estimators': 200, 'subsample': 1.0}

Tuning Random Forest...
Best Params: {'criterion': 'gini', 'max_depth': None, 'max_features': 'sqrt', 'min_samples_split': 2, 'n_estimators': 200}

Tuning Stacking...
Best Params: {'final_estimator': RandomForestClassifier()}

```

Figure 5: Visualization of hyperparameter tuning results.

Table 3: Random Forest - Best Parameters and Performance

n_estimators	Max Depth	Criterion	Accuracy	F1 Score	AUC
200	10	entropy	0.9543	0.9382	0.9895

Table 4: Stacked Ensemble - Best Parameters and Performance

Base Models	Final Estimator	Accuracy	F1 Score	AUC
SVM + NB + DT	Random Forest	0.9701	0.9570	0.9893

7. Cross-Validation Results

Table 5: 5-Fold Cross Validation Results (Accuracy Scores)

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg. Accuracy
Decision Tree	0.928	0.971	0.971	0.941	0.956	0.9532
AdaBoost	0.928	0.985	1.000	0.985	1.000	0.9796
Gradient Boosting	0.942	0.956	0.985	0.985	0.956	0.9649
XGBoost	0.942	0.985	0.985	0.985	0.985	0.9766
Random Forest	0.928	0.926	0.971	0.956	0.971	0.9502
Stacked Model	0.928	0.971	0.956	0.985	1.000	0.9679

8. Feature Importance Visuals

```
Classification Report:
              precision    recall  f1-score   support

     0       0.95         1.00         0.97         72
     1       1.00         0.90         0.95         42

 accuracy          0.96         114
 macro avg         0.97         0.95         0.96         114
weighted avg         0.97         0.96         0.96         114
```

XGBoost | Accuracy: 0.9737, F1: 0.9630, AUC: 0.9954
Confusion Matrix:

```
[[72  0]
 [ 3 39]]
Classification Report:
              precision    recall  f1-score   support

     0       0.96         1.00         0.98         72
     1       1.00         0.93         0.96         42

 accuracy          0.97         114
 macro avg         0.98         0.96         0.97         114
weighted avg         0.97         0.97         0.97         114
```

Random Forest | Accuracy: 0.9649, F1: 0.9500, AUC: 0.9942
Confusion Matrix:

```
[[72  0]
 [ 4 38]]
Classification Report:
              precision    recall  f1-score   support

     0       0.95         1.00         0.97         72
     1       1.00         0.90         0.95         42

 accuracy          0.96         114
 macro avg         0.97         0.95         0.96         114
weighted avg         0.97         0.96         0.96         114
```

Stacking | Accuracy: 0.9649, F1: 0.9512, AUC: 0.9950

Confusion Matrix:

```
[[71  1]
 [ 3 39]]
```

Figure 6: Top 15 Feature Importances.

9. Observations and Conclusions

- **Best Model: AdaBoost** achieved the highest overall performance with an average cross-validated accuracy of 98.0%. XGBoost and the Stacked Ensemble also performed very competitively, with accuracies of 97.7% and 96.8% respectively.
- **Decision Tree vs Ensembles:** The standalone Decision Tree was the weakest model with 95.3% accuracy. All ensemble methods significantly improved upon this baseline, demonstrating the power of combining multiple models to reduce variance and improve generalization.
- **Effect of Hyperparameters:** Tuning parameters like `n_estimators`, `max_depth`, and `learning_rate` was crucial. For instance, the tuned AdaBoost model significantly outperformed its default version, highlighting the importance of systematic hyperparameter optimization via methods like GridSearchCV.
- **Generalization and Stability:** Boosting methods (AdaBoost, Gradient Boosting, XGBoost) demonstrated excellent generalization with very high and stable accuracy scores across the 5 cross-validation folds. The Stacked Model also showed strong, stable performance.
- **Conclusion:** Ensemble methods, particularly boosting algorithms, are highly effective for this classification task. AdaBoost emerged as the top-performing model, showcasing the benefit of adaptively re-weighting instances to focus on difficult cases.